

Lecture 21

C Programming I

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science

College of Engineering

University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Basic constructs

C Programming Language (1972)

- Inspired the design of Python and Javascript

Minimal programming abstractions compared to Python and Elm

- “Low-level” PL: Closer to how hardware executes code
 - Most likely faster compared to Python/Elm
 - Extra work needed to provide Python/Elm conveniences

C Programming Language (1972)

Will be used in CS 32/33 and CS 20/21/140/155

- Coded in C:
 - Python: [CPython Github repo](#)
 - Git: [Git repo](#)
 - Linux kernel: [Linux kernel repo](#)

Hello World in C

Create `.c` program (`hello.c`)

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n");

    return 0;
}
```

Hello World in C

Compile `hello.c` into executable program `hello`:

```
gcc hello.c -o hello
```

Run `hello` program:

```
./hello
```

Do not forget to recompile after making changes to your code!

Hello World in C

main function is always called at start

- Returns an `int` (0 indicates success)
- `return 0` can be omitted for `main`

Built-in library `stdio.h` provides `printf`

- `#include <stdio.h>` allows use of its functions
- `printf` function prints given string on screen
 - String literals are *double-quoted*
 - Does not automatically include `\n` at the end

Hello World in C

Statements end with a semicolon

- Except for those ending in *blocks* ({}) and colons
- Preprocessor directives (#) are not statements

Variables

Declaration

```
int x;  
int y, z;
```

- Cannot use variables undeclared in scope
- Cannot redeclare variables within scope
- Variables initially contain garbage
- Naming convention: snake_case (like Python)

Variables

Assignment

```
x = 1;  
y = 2;  
z = 3;
```

- Variable must already be declared in scope

Variables

Initialization

```
int x = 1;  
int y = 2, z = 3;
```

- Initialization = Declaration + assignment

Variables

`const` modifier: cannot be reassigned; must be initialized

```
const int x;
```

```
const int y = 1;
```

```
y = 2;
```

Operators and Control Structures

A lot of operators same as in Python, but some only in C (not in Python)

```
int main() {  
    int a = (1 + (2 - 3) * 4 / 5) % 6;  
    int b = 150;  
  
    b++; // post-increment  
    b--; // post-decrement  
    b += 2;  
    b -= 100;  
}
```

```
b *= 10;  
b /= 2;  
  
int c = !((1 && 0) || !0);  
int d = (((5 | 1) & 3) ^ 5) << 3;  
int e = 1 <= 2;  
int f = 11 > 12 ? 1 : 0;  
}
```

No boolean type (see end of slides); zero is false, nonzero is true

Operators and Control Structures

if-else, while, and for

21/control_structures.c

Operators and Control Structures

- Also available: `continue, break, do {} while`

Printing Int Variables

Use `%d` in `printf` to print integers (signed 32-bit)

- `%d` in string gets substituted with corresponding argument passed after string:

```
#include <stdio.h>

int main() {
    int x = 11, y = 12;
    printf("%d + %d = %d\n", x, y, x + y);
}
```

Printing Int Variables

- `printf("%d + %d = %d\n", x, y, x + y);`
 - Prints `11 + 12 = 23`

Format specifiers can define more complex behavior; more info here: https://en.wikipedia.org/wiki/Printf#Format_specifier

Functions in C

```
<return_type> <func_name>(<arg1_type> <arg1_name>, ...) {  
    <body>  
}
```

- Must be defined outside any block (top-level)
- No anonymous and higher-order functions

Functions in C

```
int square(int n) {  
    return n * n;  
}
```

```
int sum_of_squares(int a, int b) {  
    return a * a + b * b;  
}
```

```
int main() {  
    int x = 12;  
    int y = square(x);  
}
```

```
int z = sum_of_squares(x, y);  
}
```

Functions in C

Functions must be declared prior to being called (either via prototype or definition)

Functions in C

```
// `void` type: Similar to `None` of Python,  
// but not a value; no need for `return`
```

```
void show_sum(int a, int b) {  
    printf("%d + %d = %d\n", a, b, a + b);  
} // Complete definition
```

```
int main() {  
    show_sum(1, 2); // Definition appears  
}                 // before call; valid
```


Functions in C

```
void show_sum(int, int); // Function prototype
```

```
int main() {           // Function prototype  
    show_sum(1, 2);    // appears before  
}                      // call; valid
```

```
void show_sum(int a, int b) {  
    printf("%d + %d = %d\n", a, b, a + b);  
}
```

Top Level and Scoping Rules

Top level: Region of source code outside any block

- Allowed: Declarations, preprocessor directives

Global variable: Declared in top level Local variable: Declared inside some block

- Local variable is in scope only within its block
- Usage is valid only if declaration is in enclosing block

Top Level and Scoping Rules

```
// #define is a preprocessor directive
// that replaces `N` with `140`
#define N 140

int global = 12; // Valid; variable declaration

int main() {      // Valid; function declaration
    int local = 150;

    local++;      // Valid; `local` is in same block
    global++;     // Valid; `global` has global scope
```

```
if (local == global) { // Valid; same as above
    int inner = 21;
    inner--; // Valid; `inner` is in same block
    local--; // Valid; `local` in enclosing block
    global--; // Valid; `global` has global scope
}

for (int i = 0; i < N; i++) { // Special case: init
    global += i; // var of `for` can be
} // used within `for` ()
}
```

Top Level and Scoping Rules

```
#include <stdio.h>

int main() {
    for (int k = 0; k < 12; k++) {
        f(); // Invalid; `f` has not yet been declared
    }
    k++; // Invalid; `k` is not in scope
}

void f() {
    // Invalid; function definition inside block
}
```

```
void g() {  
    printf("Hello, world!\n");  
}  
}
```

```
// Invalid; only declarations and directives in top level  
if (k == 12) { // Invalid; `n` is not in scope  
    printf("Hello, world!\n");  
}
```

Data Types

Integer Data Types

C integers are either

- signed (allows negative numbers), or
- unsigned (only nonnegatives)

and have size-based limits

Note: The following table below assumes 64-bit GCC (types in blue may differ for other platforms)

Integer Data Types

<u>Bits(n)</u>	<u>Signed</u>	<u>Unsigned</u>
8 (1 byte)	<code>char</code> and <code>int8_t</code> (-128 to 127)	<code>unsigned char</code> and <code>uint8_t</code> (0 to 255)
32 (4 bytes)	<code>int</code> and <code>int32_t</code> (-2147483648 to 2147483647)	<code>unsigned int</code> and <code>uint32_t</code> (0 to 4294967295)

<u>Bits(n)</u>	<u>Signed</u>	<u>Unsigned</u>
64 (8 bytes)	<code>long long</code> , <code>ssize_t</code> , and <code>int64_t</code> $([-2^{63}, 2^{63} - 1])$	<code>unsigned long long</code> , <code>size_t</code> , and <code>uint64_t</code> $([0, 2^{64} - 1])$

Integer Data Types

`stdint.h` provides `uint8_t`, `uint32_t`, and `uint64_t`

- 16 bits (uncommon): `short`, `int16_t`, etc.
- Need larger integers? Define them yourself (CS 32)

Arrays in C

C arrays are *zero-indexed* and *fixed-size* (contains exactly N elements)

```
#include <stdio.h>

int main() {
    // `nums` contains five integers; N = 5
    int nums[5] = {10, 20, 30, 40, 50};

    for (int i = 0; i < 5; i++) {
        nums[i] *= nums[i];
    }
}
```

```
}
```

```
for (int i = 0; i < 5; i++) {  
    printf("nums[%d]: %d\n", i, nums[i]);  
}
```

```
}
```

Arrays in C

- `int nums[5] = {};` initializes all five to 0
- `int nums[5] = {10, 20};` sets last three to 0
- `int nums[5];` has elements set to garbage values!
- `int nums[] = {10, 20};` has inferred size of 2
- `int nums[] = {};` makes little sense (size is 0)
 - C arrays cannot expand once declared

Arrays in C

Limitation: cannot ask for current element count

- Have to keep track manually

Arrays in C

21/arrays.c

Issue with Array Bounds

*Warning: Program will not crash on out-of-bounds reads/writes;
harder to catch and may lead to subtle bugs*

Issue with Array Bounds

```
#include <stdio.h>

int main() {
    int nums[] = {10, 20, 30};
    int arr[5] = {};
    int sum = 0;

    for (int i = 0; i < 7; i++) {
        arr[i] = 12000;
    }
}
```

```
for (int i = 0; i < 3; i++) {  
    sum += nums[i];  
}  
  
printf("sum is %d\n", sum);  
}
```

Issue with Array Bounds

C does not have Python's `for ... in`; must be extra careful with iteration and bounds

Nested Arrays

Array can be arbitrarily nested like in Python

- 2D array declaration: `<type> <name>[rows][cols];`

Nested Arrays

```
#include <stdio.h>

int main() {
    int matrix[2][3] = {
        {10, 20, 30},
        {40, 50, 60},
    };

    matrix[1][2] *= 2;

    printf("%d\n", matrix[0][0]);    // 10
```

```
printf("%d\n", matrix[0][1]);    // 20
printf("%d\n", matrix[1][2]);    // 120

int other[3][4] = {};           // Init all to 0
printf("%d\n", other[2][3]);     // 0

int garbage[3][4];              // Contains garbage
printf("%d\n", garbage[2][3]);   // Garbage value
}
```

Review: ASCII Encoding Scheme

- Valid ASCII values range from 0 to 127 (note that 32 is the space character)
- The string "Hello X" as array of integers: {72, 101, 108, 108, 111, 32, 88}

Review: ASCII Encoding Scheme

ASCII

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
0_	U+0000	U+0001	U+0002	U+0003	U+0004	U+0005	U+0006	U+0007	U+0008	U+0009	U+000A	U+000B	U+000C	U+000D	U+000E	U+000F
	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1_	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	U+0010	U+0011	U+0012	U+0013	U+0014	U+0015	U+0016	U+0017	U+0018	U+0019	U+001A	U+001B	U+001C	U+001D	U+001E	U+001F
2_	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
3_	U+0020	U+0021	U+0022	U+0023	U+0024	U+0025	U+0026	U+0027	U+0028	U+0029	U+002A	U+002B	U+002C	U+002D	U+002E	U+002F
	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
4_	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
	U+0030	U+0031	U+0032	U+0033	U+0034	U+0035	U+0036	U+0037	U+0038	U+0039	U+003A	U+003B	U+003C	U+003D	U+003E	U+003F
5_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
6_	U+0040	U+0041	U+0042	U+0043	U+0044	U+0045	U+0046	U+0047	U+0048	U+0049	U+004A	U+004B	U+004C	U+004D	U+004E	U+004F
	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
7_	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
	U+0050	U+0051	U+0052	U+0053	U+0054	U+0055	U+0056	U+0057	U+0058	U+0059	U+005A	U+005B	U+005C	U+005D	U+005E	U+005F
8_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95
9_	U+0060	U+0061	U+0062	U+0063	U+0064	U+0065	U+0066	U+0067	U+0068	U+0069	U+006A	U+006B	U+006C	U+006D	U+006E	U+006F
	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
A_	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111
	U+0070	U+0071	U+0072	U+0073	U+0074	U+0075	U+0076	U+0077	U+0078	U+0079	U+007A	U+007B	U+007C	U+007D	U+007E	U+007F
B_	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL
	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127

C-style Strings

Null-terminated string: Array of `char` that ends in a null character

- `%c` format specifier: ASCII symbol equivalent of `char`
- `%s` format specifier: `char` array as null-terminated string

C-style Strings

```
#include <stdio.h>

int main() {
    char str1[] = {72, 101, 108, 108, 111, 32, 88, 0};
    int i = 0;

    for (int i = 0; str1[i] != 0; i++) {
        printf("str1[%d]: %d (%c)\n", i, str1[i], str1[i]);
    }

    char str2[] = {'H', 'e', 'l', 'l', 'o', ' ', 'X', '\0'};
```

```
char str3[] = "Hello X";           // \0 is null character
```

```
printf("str1 as string: %s\n", str1);
```

```
printf("str2 as string: %s\n", str2);
```

```
printf("str3 as string: %s\n", str3);
```

```
}
```

C-style Strings

- Single-quoted character is `int`; can be coerced into `char`
- Double-quoted string is shorthand for array of `chars`
 - `\0` is automatically appended at end of string

Example

- Disregarding data type limits, what is the expected output of the code below?
- How about when considering limits?

```
21/example.c
```

Staying within Data Type Limits

- Approach 1: Algebraic manipulation (if possible)

```
uint32_t middle_int(uint32_t x, uint32_t y) {  
    uint32_t min = x < y ? x : y;  
    uint32_t max = x > y ? x : y;  
  
    return min + (max - min) / 2;  
}
```

Staying within Data Type Limits

Self-check: Why is $x / 2 + y / 2$ not always correct for this problem?

- float is 32-bit floating-point number; double is 64-bit
- Type casting allows conversion from integers to floating points
- Be careful with floating points (CS 11)

Staying within Data Type Limits

- Approach 2: Type-cast operands to wider type

```
uint32_t middle_int(uint32_t x, uint32_t y) {  
    // `x` and `y` are type-casted  
    // to `uint64_t`  
    uint64_t a = x; // Implicit cast  
    uint64_t b = (uint64_t) y; // Unneeded  
  
    return (a + b) / 2; // `a + b` portion  
                        // is `uint64_t`  
}
```

Staying within Data Type Limits

```
uint32_t middle_int(uint32_t x, uint32_t y) {  
    // One-liner  
    return (((uint64_t) x) + ((uint64_t) y)) / 2;  
}
```

Note on return value: `uint64_t` result gets coerced into `uint32_t` return type; should be fine since possible values fit in `uint32_t` for this problem

Reading assignment

C Compiler Installation

GCC installation

- GCC and Clang are acceptable compilers for CS 12 25.2
- Linux should have gcc(`sudo apt -y install gcc` if not)
- macOS has clang via `xcode-select --install` (`brew install gcc` alternatively)
- Windows: Instructions to use gcc via WSL

stdbool.h

- stdbool.h can be included for “fake” booleans:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    bool x = true;
    bool y = false;

    printf("%d %d\n", x, y); // 1 0
}
```

Format specifiers for integer types

<u>Signed type</u>	<u>Format specifier</u>	<u>Signed type</u>	<u>Format specifier</u>
char	%hhd	ssize_t	%zd
short	%hd	int8_t	"%" PRIu8
int	%d	int16_t	"%" PRIu16
long	%ld	int32_t	"%" PRIu32
long long	%lld	int64_t	"%" PRIu64

Format specifiers for integer types

<u>Unsigned type</u>	<u>Format specifier</u>	<u>Unsigned type</u>	<u>Format specifier</u>
unsigned char	%hhu	size_t	%zu
unsigned short	%hu	uint8_t	"%" PRIu8
unsigned int	%u	uint16_t	"%" PRIu16
unsigned long	%lu	uint32_t	"%" PRIu32

<u>Unsigned type</u>	<u>Format specifier</u>	<u>Unsigned type</u>	<u>Format specifier</u>
unsigned long long	%llu	uint64_t	"%" PRIu64

Format specifiers for integer types

- Change `d` or `u` above to `x` for lowercase hex and `X` for uppercase hex
- Note: `inttypes.h` include needed for `stdint.h` types